Exercices Python — Jour 5 : structures conditionnelles avancées, gestion des erreurs et boucles

Exercices — Python Jour 5

Conditions avancées, gestion des erreurs et boucles

Contexte pédagogique : ces exercices portent sur les notions vues dans le cours *« Structures conditionnelles avancées, gestion des erreurs et boucles »* (Jour 5) :

- **Conditions avancées** : conditions imbriquées, opérateurs logiques (`and`, `or`, `not`), opérateur ternaire, `match / case` approfondi (motifs de type `case int()`, `case str()`, gardes `case ... if ...`, cas par défaut `case _`) ;
- **Gestion des exceptions** : `try / except / else / finally`, plusieurs `except` enchaînés, `raise`, exceptions courantes (`ValueError`, `TypeError`, `ZeroDivisionError`, `IndexError`, `KeyError`, `AttributeError`...) ;
- **Mots clés spéciaux** : `break`, `continue`, `pass` ;
- **Boucles** : `while`, `for`, `range(start, stop, step)`, boucles imbriquées, `for / else` ;
- **Parcours de structures** : `for` sur `list`, `tuple`, `str`, `set`, `dict` (`.items()`, `.keys()`, `.values()`) ;
- **Compréhensions de liste** : `[expression for element in sequence if condition]` ;
- **Fonctions (`def`)** : paramètres, valeurs de retour (`return`), paramètres avec valeur par défaut, docstring — utilisées dans le cours dès les premiers exemples pour illustrer le `match / case` et les exceptions.

Notions déjà acquises (Jour 4) qui restent autorisées : méthodes `str` (`split()`, `strip()`, `upper()`, `lower()`, `isdigit()`, `isalnum()`...), `list`, `tuple`, `set`, `dict`, `len()`, `int()`.

Règle du jeu : on reste concentré sur les notions ci-dessus. Pas de compréhensions de `dict` ou de `set` (seule la compréhension de **liste** a été vue), pas d'expressions régulières, pas de programmation orientée objet (classes) : ces notions seront vues plus tard dans la formation.

Consigne générale de rendu : pour chaque exercice, écrivez un script Python (avec ou sans fonction `def`, selon ce qui est demandé) qui déclare vous-même les variables de test indiquées, exécute le traitement demandé, et affiche des messages clairs avec `print()` / f-strings. Testez bien **toutes** les valeurs du jeu d'essai fourni.

Exercice 1 ★

Triage d'alertes de supervision

Contexte. Un centre de supervision reçoit des alertes composées de trois informations : une gravité, l'état d'activité du service concerné et un indicateur de maintenance. Malheureusement, le capteur qui transmet la gravité n'est pas toujours fiable : il arrive qu'il envoie un texte au lieu d'un nombre, ou une valeur aberrante. Vous devez trier ces alertes avant de décider d'une action.

Jeu d'essai. Chaque alerte est un tuple `(gravite, service_actif, en_maintenance)` :

```
alertes = [
    (5, True, False),
    ("2", True, False),
    (4, False, False),
    (3, True, True),
    (-1, True, False),
]
```

Consignes. Pour chaque alerte `(gravite, service_actif, en_maintenance)` :

1. Utilisez `match gravite:` avec des motifs de **type** (comme dans le cours) pour d'abord écarter les gravités invalides :
   - `case int() if gravite >= 1 and gravite <= 5` : la gravité est valide, poursuivez avec les étapes 2 à 4 **à l'intérieur de ce cas**.
   - `case int()` (un entier, mais hors de 1-5) : affichez `"Alerte ignoree : gravite <gravite> hors plage (1-5)"`.
   - `case str()` (la gravité est un texte) : affichez `"Alerte ignoree : gravite recue en texte ('<gravite>')"`.
   - `case _` (tout autre type) : affichez `"Alerte ignoree : type de gravite non reconnu"`.
2. **Pour les alertes de gravité valide uniquement**, utilisez une condition imbriquée : si `en_maintenance` est vrai, affichez `"Alerte (gravite=<gravite>) -> suspendue (maintenance en cours)"`.
3. Sinon (pas en maintenance), imbriquez un second niveau : si `not service_actif`, alors `priorite = "CRITIQUE_HORS_SERVICE"` ; sinon, déterminez `priorite` avec `if / elif / else` selon la gravité : `>= 5` → `"CRITIQUE"`, `>= 3` → `"MAJEURE"`, sinon → `"MINEURE"`.
4. Calculez, avec l'opérateur **ternaire**, `action_immediate = "Oui"` si `priorite` vaut `"CRITIQUE"` **ou** `"CRITIQUE_HORS_SERVICE"`, sinon `"Non"`. Affichez `"Alerte (gravite=<gravite>) -> priorite=<priorite>, action_immediate=<action_immediate>"`.

Contraintes techniques obligatoires. `match / case` avec motifs de type et garde (`if` dans le `case`), condition imbriquée sur **au moins 2 niveaux**, opérateurs logiques (`and`, `not`, `or`), opérateur ternaire.

Résultat attendu (exemple, vérifié à l'exécution).

```
Alerte (gravite=5) -> priorite=CRITIQUE, action_immediate=Oui
Alerte ignoree : gravite recue en texte ('2')
Alerte (gravite=4) -> priorite=CRITIQUE_HORS_SERVICE, action_immediate=Oui
Alerte (gravite=3) -> suspendue (maintenance en cours)
Alerte ignoree : gravite -1 hors plage (1-5)
```

Bonus (facultatif). Ajoutez un compteur `nb_ignorees` qui s'incrémente à chaque fois qu'une alerte est écartée (gravité invalide), et affichez-le à la fin.

Exercice 2 ★★

Lecture sécurisée d'un fichier de configuration réseau

Contexte. Un fichier de configuration réseau, une fois lu, est représenté par un dictionnaire de chaînes de caractères (toutes les valeurs sont du texte brut, à convertir). Certaines clés attendues peuvent être absentes, certaines valeurs peuvent être mal formées, et certaines valeurs numériques peuvent être hors de la plage autorisée. Vous devez écrire une fonction de lecture robuste qui ne fait jamais planter le programme.

Jeu d'essai.

```
config_brute = {
    "port": "8080",
    "timeout": "30",
    "max_connexions": "cent",
}
```

Consignes.

1. Écrivez une fonction `lire_parametre(config, cle, mini, maxi)` qui :
   (a) dans un bloc `try`, récupère `config[cle]` (qui peut lever `KeyError` si la clé est absente), la convertit avec `int()` (qui peut lever `ValueError` si le texte n'est pas un nombre), puis vérifie que la valeur est comprise entre `mini` et `maxi` — si ce n'est pas le cas, **déclenchez vous-même** une `ValueError` avec `raise` et un message précisant la clé, la valeur et la plage attendue ;
   (b) intercepte séparément `KeyError` (affiche `"Erreur : la cle '<cle>' est absente de la configuration."`) et `ValueError` (affiche `"Erreur : <message de l'exception>"`) — dans les deux cas, la fonction renvoie `None` ;
   (c) dans un bloc `else` (exécuté uniquement si **aucune** erreur ne s'est produite), affiche `"OK : <cle> = <valeur>"` et renvoie `valeur` ;
   (d) dans un bloc `finally`, affiche **systématiquement** `"Tentative de lecture de '<cle>' terminee."`, que la lecture ait réussi ou échoué.
2. Appelez cette fonction pour les 4 couples `(cle, mini, maxi)` suivants, dans cet ordre : `("port", 1, 65535)`, `("timeout", 1, 120)`, `("max_connexions", 1, 1000)`, `("backlog", 1, 100)`.

Contraintes techniques obligatoires. fonction `def` avec docstring, `try / except / else / finally`, au moins 2 `except` différents, `raise` déclenché manuellement.

Résultat attendu (exemple, vérifié à l'exécution).

```
OK : port = 8080
Tentative de lecture de 'port' terminee.
OK : timeout = 30
Tentative de lecture de 'timeout' terminee.
Erreur : invalid literal for int() with base 10: 'cent'
Tentative de lecture de 'max_connexions' terminee.
Erreur : la cle 'backlog' est absente de la configuration.
Tentative de lecture de 'backlog' terminee.
```

Remarque pédagogique. Pour `"max_connexions"`, c'est l'appel `int("cent")` lui-même qui lève la `ValueError` (elle n'est pas déclenchée par votre `raise`) : le message affiché est donc celui produit **directement par Python**, et non celui que vous auriez rédigé pour la vérification de plage. Les deux passent par le **même** `except ValueError`, ce qui est exactement l'intérêt de cette structure.

Bonus (facultatif). Ajoutez un cinquième appel avec une plage volontairement absurde, par exemple `("port", 1, 65535)` mais avec `config_brute["port"] = "99999"` dans une configuration copiée, pour déclencher **votre** `raise` (hors plage) plutôt que celui de `int()`.

Exercice 3 ★★

Scan de ports avec arrêt anticipé et filtrage

Contexte. Un scanner de sécurité doit parcourir une liste de ports bruts (parfois invalides), ignorer certains ports « en watchlist » dont le traitement spécifique n'est pas encore implémenté, et s'arrêter dès qu'il détecte le port SSH (22) ouvert.

Jeu d'essai.

```
ports_watchlist = {3389}
ports_a_scanner = [0, 80, -5, 443, 3389, 99999, 22, 8080]
```

Consignes. Parcourez `ports_a_scanner` avec une boucle `for` :

1. Si le port est **invalide** (`<= 0` ou `> 65535`), affichez `"Port <port> : invalide, ignore"` et passez immédiatement au port suivant avec `continue` (sans exécuter le reste du corps de la boucle pour ce port).
2. Si le port fait partie de `ports_watchlist`, utilisez `pass` : c'est un traitement spécifique **pas encore implémenté**, qui ne doit **rien** faire de particulier ici (observez bien, dans le résultat attendu ci-dessous, que le port continue malgré tout à être traité par la suite du code : `pass` ne fait **que** satisfaire la syntaxe, contrairement à `continue` qui aurait sauté la suite).
3. Si le port vaut `22`, affichez `"Port 22 (SSH) : trouve, arret du scan."` puis quittez **immédiatement** la boucle avec `break` : les ports suivants dans la liste ne doivent **plus du tout** être examinés.
4. Pour tous les autres ports valides, affichez `"Port <port> : scanne, rien de particulier a signaler."`

5. Après la boucle (qu'elle se soit terminée par un `break` ou naturellement), affichez `"Scan termine ou interrompu."`

Contraintes techniques obligatoires. `continue`, `pass`, `break`, boucle `for`, opérateur `in` sur un `set`.

Résultat attendu (exemple, vérifié à l'exécution).

```
Port 0 : invalide, ignore
Port 80 : scanne, rien de particulier a signaler.
Port -5 : invalide, ignore
Port 443 : scanne, rien de particulier a signaler.
Port 3389 : scanne, rien de particulier a signaler.
Port 99999 : invalide, ignore
Port 22 (SSH) : trouve, arret du scan.
Scan termine ou interrompu.
```

Remarque pédagogique. Le port `8080`, dernier de la liste, n'apparaît **jamais** dans le résultat : le `break` déclenché sur le port `22` arrête la boucle avant qu'il ne soit atteint. C'est la différence fondamentale avec `continue`, qui n'aurait sauté que **l'itération** du port `22` lui-même, sans arrêter le reste du scan.

Bonus (facultatif). Comptez, avec un compteur manuel incrémenté uniquement dans la branche des ports valides (ni invalides, ni celui qui déclenche le `break`), le nombre de ports effectivement « scannés normalement », et affichez-le après le message final.

Exercice 4 ★★

Simulateur de ping avec abandon après N tentatives

Contexte. Un script de supervision teste la joignabilité de plusieurs hôtes par une série de tentatives de ping simulées (chaque tentative renvoie `True` en cas de succès, `False` en cas d'échec). Il doit abandonner après un nombre maximum de tentatives si aucune n'a réussi, et s'arrêter dès la première réussite.

Jeu d'essai.

```
max_tentatives = 4
hotes = {
    "srv-web": [False, False, True, False],
    "srv-db":  [False, False, False, False],
}
```

Consignes. Pour chaque hôte (parcouru avec `.items()`) :

1. Initialisez `tentative = 0` et `succes = False`.
2. Écrivez une boucle `while tentative < max_tentatives:` qui, à chaque tour, lit la réponse simulée correspondante dans la liste de l'hôte, incrémente `tentative`, et affiche `"Tentative <tentative>/<max_tentatives> : <succes ou echec>"`.
3. Si la réponse est un succès, passez `succes` à `True` et quittez la boucle `while` avec `break` (inutile de continuer à tester).

4. Après la boucle `while`, affichez selon `succes` : `"<hote> : joignable (en <tentative> tentative(s))."` ou `"<hote> : injoignable apres <max_tentatives> tentatives, abandon."`

Contraintes techniques obligatoires. boucle `while` avec condition d'arrêt claire, `break`, compteur incrémenté manuellement, `dict.items()`.

Résultat attendu (exemple, vérifié à l'exécution).

```
--- Ping vers srv-web ---
  Tentative 1/4 : echec
  Tentative 2/4 : echec
  Tentative 3/4 : succes
srv-web : joignable (en 3 tentative(s)).
--- Ping vers srv-db ---
  Tentative 1/4 : echec
  Tentative 2/4 : echec
  Tentative 3/4 : echec
  Tentative 4/4 : echec
srv-db : injoignable apres 4 tentatives, abandon.
```

Bonus (facultatif). Construisez une liste `hotes_injoignables = []` dans laquelle vous ajoutez (avec `.append()`) le nom de chaque hôte resté injoignable, puis affichez cette liste à la toute fin du script.

Exercice 5 ★★★

Génération d'un plan d'adressage VLAN

Contexte. Vous devez générer automatiquement un plan d'adressage IP pour plusieurs VLAN, chacun avec ses premières adresses hôtes, sans rien écrire « en dur » : tout doit être calculé à partir de quelques paramètres, à l'aide de boucles imbriquées et de `range()`.

Jeu d'essai.

```
vlan_debut = 10
nb_vlans = 3
pas_vlan = 10
hotes_par_vlan = 3
```

Consignes.

1. Construisez `vlan_ids = range(vlan_debut, vlan_debut + nb_vlans * pas_vlan, pas_vlan)` (un `range` avec un **pas**), et affichez sa conversion en liste avec `list(vlan_ids)` pour vérifier les identifiants générés.
2. Avec une boucle `for` **externe** parcourant `vlan_ids`, affichez d'abord `"VLAN <vlan_id> :"`.
3. Avec une boucle `for` **interne** (`for h in range(1, hotes_par_vlan + 1)`), affichez, pour chaque VLAN, les `hotes_par_vlan` premières adresses hôtes au format `" 192.168.<vlan_id>.<h>"`.

Contraintes techniques obligatoires. `range(start, stop, step)`, boucles `for` imbriquées

(une boucle externe et une boucle interne), `list(range(...))`.
Résultat attendu (exemple, vérifié à l'exécution).

```
VLANs generes : [10, 20, 30]
VLAN 10 :
  192.168.10.1
  192.168.10.2
  192.168.10.3
VLAN 20 :
  192.168.20.1
  192.168.20.2
  192.168.20.3
VLAN 30 :
  192.168.30.1
  192.168.30.2
  192.168.30.3
```

Bonus (facultatif). Modifiez `hotes_par_vlan` pour qu'il soit **différent** d'un VLAN à l'autre (par exemple en le stockant dans une liste parallèle `[3, 5, 2]`) et adaptez la boucle interne en conséquence à l'aide d'`enumerate(vlan_ids)`.

Exercice 6 ★★★

Recherche d'une signature d'attaque et extraction condensée

Contexte. Un journal de connexions doit être scruté à la recherche d'une signature précise (par exemple un mot-clé associé à une attaque connue). Si elle n'est trouvée nulle part, il faut le signaler explicitement — c'est le cas d'usage typique du bloc `for / else` de Python.
Jeu d'essai.

```
logs = [
    "10:01:02 INFO 192.168.1.5 connexion etablie",
    "10:02:15 INFO 192.168.1.9 requete traitee",
    "10:05:41 WARNING 192.168.1.5 tentative repetee",
    "10:07:03 INFO 192.168.1.12 requete traitee",
    "10:09:59 INFO 192.168.1.5 deconnexion",
]
signature_recherchee = "ERROR"
```

Consignes.

1. Écrivez une boucle `for ligne in logs:` qui teste, avec l'opérateur `in`, si `signature_recherchee` apparaît dans la ligne. Si oui, affichez `"Signature '<signature>' detectee : <ligne>"` puis quittez la boucle avec `break`.
2. Ajoutez un bloc `else` **rattaché à ce** `for` : il doit afficher `"Aucune occurrence de la signature '<signature>' dans le journal."` — et ne s'exécutera, comme vu en cours, que si la boucle s'est terminée **sans** `break`.
3. À l'aide d'une **compréhension de liste**, construisez `ips_warning` contenant, pour chaque

ligne du journal contenant `"WARNING"`, le troisième mot de la ligne (l'adresse IP), obtenu avec `ligne.split()[2]`.

4. À l'aide d'une seconde compréhension de liste (**sans** condition cette fois), construisez `niveaux` contenant le niveau (deuxième mot, `ligne.split()[1]`) de **chaque** ligne du journal.
5. Affichez `ips_warning` et `niveaux`.

Contraintes techniques obligatoires. `for / else`, `break`, opérateur `in`, deux compréhensions de liste (une avec condition, une sans).

Résultat attendu (exemple, vérifié à l'exécution).

```
Aucune occurrence de la signature 'ERROR' dans le journal.
IPs en WARNING : ['192.168.1.5']
Niveaux rencontres : ['INFO', 'INFO', 'WARNING', 'INFO', 'INFO']
```

Remarque pédagogique. `signature_recherchee = "ERROR"` est **volontairement** absente de `logs` : c'est le seul moyen de vérifier que votre bloc `else` associé au `for` fonctionne réellement (si vous ne voyez jamais ce message s'afficher, même en l'absence de la signature, c'est que votre `else` est mal rattaché à la boucle).

Bonus (facultatif). Recommencez la recherche avec `signature_recherchee = "WARNING"` (présente cette fois) et vérifiez que le message `"Signature détectée"` s'affiche bien à la place du message du `else`.

Exercice 7 ★★★★

Synthèse : pare-feu anti brute-force avec supervision

Contexte. Ce dernier exercice combine **toutes** les notions du jour : une fonction protège un service contre les attaques par force brute (comme le fait un outil tel que *fail2ban*), en gérant à la fois des tentatives mal formées (erreurs de journalisation) et un nombre maximal d'échecs, puis un `match / case` interprète le résultat pour chaque adresse IP.

Jeu d'essai.

```
connexions_brutes = {
    "192.168.1.10": ["Motdepasse1", " SuperSecret123 ", "azerty"],
    "192.168.1.20": ["azerty", 123456, "password", "qwerty"],
}
mot_de_passe_correct = "supersecret123"
```

Remarque. Observez que, pour `"192.168.1.20"`, la deuxième tentative (`123456`) est un **entier** et non une chaîne : c'est une erreur de journalisation simulée, qu'il faudra détecter et ignorer proprement plutôt que de laisser planter le script.

Consignes.

1. Écrivez une fonction `tenter_connexion(ip, tentatives, mot_de_passe_correct, max_echecs=3)` (notez le paramètre `max_echecs` avec sa **valeur par défaut**) qui :
   (a) initialise `echecs = 0` et `index = 0` ;
   (b) boucle avec `while index < len(tentatives) and echecs < max_echecs:` ;

(c) à chaque tour, récupère la tentative courante puis incrémente `index` ;
(d) dans un bloc `try`, normalise la tentative avec `.strip().lower()` ; si cela lève une `AttributeError` (la tentative n'est pas une chaîne), affiche `" Tentative ignoree (format invalide : <tentative>)."` (utilisez `!r` dans le f-string pour afficher la valeur brute) et passe au tour suivant avec `continue` ;
(e) si la tentative normalisée est égale à `mot_de_passe_correct`, affiche `" Connexion reussie pour <ip>."` et `return "connectee"` immédiatement ;
(f) sinon, incrémente `echecs` et affiche `" Echec <echecs>/<max_echecs> pour <ip>."` ;
(g) après la boucle `while`, si `echecs >= max_echecs`, affiche `" IP <ip> BLOQUEE."` et renvoie `"bloquee"` ; sinon renvoie `"inconnue"`.

2. Pour chaque IP de `connexions_brutes` (parcourue avec `.items()`), affichez `"--- <ip> ---"`, appelez la fonction, puis utilisez `match` sur le résultat renvoyé pour afficher un message final : `"connectee"` → `" Statut final : acces autorise."`, `"bloquee"` → `" Statut final : IP bloquee, alerte generee."`, cas par défaut → `" Statut final : tentatives insuffisantes pour conclure."`

Contraintes techniques obligatoires. `def` avec paramètre à valeur par défaut, `while` avec condition composée (`and`), `try / except AttributeError`, `continue`, `return` anticipé, `match / case`, `dict.items()`.

Résultat attendu (exemple, vérifié à l'exécution).

```
--- 192.168.1.10 ---
  Echec 1/3 pour 192.168.1.10.
  Connexion reussie pour 192.168.1.10.
  Statut final : acces autorise.
--- 192.168.1.20 ---
  Echec 1/3 pour 192.168.1.20.
  Tentative ignoree (format invalide : 123456).
  Echec 2/3 pour 192.168.1.20.
  Echec 3/3 pour 192.168.1.20.
  IP 192.168.1.20 BLOQUEE.
  Statut final : IP bloquee, alerte generee.
```

Remarque pédagogique. Pour `"192.168.1.20"`, la tentative invalide (`123456`) est ignorée par le `continue` **sans** incrémenter `echecs` : elle ne compte donc pas comme un échec de mot de passe, seulement comme une anomalie de journalisation. Ce sont bien les 3 **autres** tentatives (`"azerty"`, `"password"`, `"qwerty"`) qui atteignent `max_echecs` et déclenchent le blocage.

Bonus (facultatif). Construisez, à l'aide d'une compréhension de liste, la liste des IP dont le statut final est `"bloquee"`, et affichez-la à la toute fin du script.

Critères de réussite (auto-évaluation)

Avant de considérer un exercice comme terminé, vérifiez que :

- votre script fonctionne sur **toutes** les valeurs du jeu d'essai fourni, y compris les cas « pièges

» (types inattendus, valeurs hors plage, tentatives mal formées) ;

- vous distinguez correctement les rôles de `break`, `continue` et `pass`, et ceux de `except`, `else` et `finally` ;
- vous n'utilisez **que** des notions vues jusqu'au Jour 5 inclus (pas de compréhensions de `dict`/`set`, pas de classes, pas de regex) ;
- chaque méthode ou structure listée dans les « contraintes techniques obligatoires » de l'exercice apparaît bien au moins une fois dans votre code ;
- les messages affichés sont clairs et correspondent exactement à ce qui est demandé.